

Projektuppgift

Programmering i TypeScript (DT208G)

Titel

Öppna Akademin

Ziad Adeleo



Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.

Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.

Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Institutionen för data och elektroteknik(DET)

Författare: Ziad Adeleo, ziba2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Öppna Akademin är en fiktiv webbplats för gratis onlineutbildning baserad på svensk universitetsstandard. Webbapplikationen utvecklades med Angular version 21 och består av fem webbsidor och en 404-sida för hantering av felaktiga URL:er.

Projektets syfte var att utveckla en responsiv och användarvänlig webbplats med funktionalitet för sökning och sortering av kurser samt hantering av ett personligt ramschema.

Webbplatsen utvecklades med komponentbaserad arkitektur, routing, services och localStorage för att skapa en strukturerad och modulär lösning.

Resultatet blev en singel page application där användaren kan navigera mellan webbplatsens olika sidor, söka och sortera kurser och hantera sitt ramschema.

Innehållsförteckning

Sammanfattning	iii
1 Introduktion	6
1.1 Bakgrund och problemmotivering	6
1.2 Övergripande syfte	6
1.3 Avgränsningar	7
1.4 Detaljerad problemformulering	8
2 Teori	9
2.1 Angular	9
2.1.1 Komponenter	9
2.1.2 Data binding	9
2.1.3 Services och dependency injection	10
2.1.4 HTTP	10
2.1.5 Routing	10
2.2 Övriga programvaror	10
2.2.1 Visual studio code	10
2.2.2 Squoosh	10
2.2.3 Remove.bg	11
3 Metod	11
3.1 Förarbete	12
3.1.1 Språkstil och ton	12
3.1.2 design	13
3.1.3 Definiering av innehåll	13
3.1.4 Bilder och ikoner	13
3.2 Planering av struktur och design	13
3.2.1 Responsiv design	14
3.2.2 Mappstruktur	14
3.2.3 Komponentstruktur	14
3.2.4 Interface	15
3.3 Planering av funktionalitet	15
3.3.1 Definiering av funktionalitet	15
3.3.2 Services och localStorage	16
3.4 Arbetsmetod	16
3.5 Versionshantering	16
4 Konstruktion	17
4.1 Routing och applikationsstruktur	17
4.2 Komponentbaserad arkitektur	17

4.3 Services och datahantering	17
4.4 Funktionalitet (filtrering, sortering och paginering)	17
4.5 Ramschema och beräkningar	18
4.6 Datamodeller och typning	18
4.7 Responsiv design	18
5 Resultat	19
6 Slutsatser	19
Källförteckning	21

1 Introduktion

Denna sektion beskriver det problem som det fiktiva universitetet Öppna Akademin vill lösa, inom ramen för projektuppgiftens krav. Projektet genomförs med vissa begränsningar och med möjlighet att inkludera extra funktionalitet utöver grundkraven.

1.1 Bakgrund och problemmotivering

Cirka 20 % av världens länder erbjuder sina medborgare, eller inom politiska och ekonomiska unioner såsom EU, kostnadsfri universitetsutbildning. Denna andel minskar ytterligare när man ser till möjligheten att läsa bredare ämnesområden och genomföra kandidat- och masterutbildningar online.

I dagsläget är de flesta universitetsutbildningar fortfarande platsbundna. Detta innebär att människor som saknar möjlighet att flytta, eller som bor i mindre ekonomiskt utvecklade regioner, inte ges möjlighet till högre utbildning.

Enligt "University of the People" har samhällen med högre utbildningsnivå positiva effekter på samhällsutvecklingen, bland annat genom ökad ekonomisk stabilitet, lägre kriminalitet och ökad jämlikhet.

Öppna Akademin är ett fiktivt universitet som erbjuder kostnadsfri onlineutbildning för alla, oavsett geografisk plats, med ambitionen att hålla svensk universitetsstandard, erbjuda ett brett kursutbud med möjligheten att skapa egen examen [1,2,3].

1.2 Övergripande syfte

Det övergripande syftet med Öppna Akademin är att skapa en digital närvaro för ett fiktivt universitet som erbjuder utbildning på universitetsnivå online.

Webbplatsen ska vara responsiv och anpassad för olika skärmstorlekar och består av följande fem huvudsidor samt en 404-sida:

- Hem
- Kurser
- Ramschema
- Om
- Kontakt
- 404

1.3 Avgränsningar

Webbplatsen är avgränsad till att uppfylla de grundläggande kraven i projektuppgiften och extra funktionalitet som kan ses som en delmängd av en fullt utvecklad universitetswebbplats.

Detta innebär att funktioner såsom autentisering, studentkonto eller mer omfattande gränssnitt som uppfyller ett riktigt universitet inte är möjligt.

1.4 Detaljerad problemformulering

Webbplatsen ska utvecklas som en responsiv applikation med Angular version 21 och uppfylla följande krav:

Samtliga sidor ska innehålla en global navigering med logotyp och länkar till webbplatsens huvudsidor och en gemensam sidfot med kontaktinformation och viktiga länkar (fiktiva).

Hem-sidan ska innehålla:

- En hero-sektion som beskriver Öppna Akademin och innehåller en call-to-action och en passande bild.
- En introduktions-sektion som beskriver vad universitetet erbjuder, kompletterad med en bild som förstärker budskapet
- En funktions-sektion som beskriver vad användaren kan göra på plattformen. Varje delsektion kompletteras med ikoner för ökad visuell tydlighet
- CTA-sektion som uppmanar användaren till interaktion
- En trovärdighets-sektion som ska till stärka den akademiska trovärdigheten

Kurser-sidan ska innehålla funktionalitet för:

- Filtrering baserat på kurskod eller kursnamn
- Filtrering baserat på ämnesområde
- Sortering i stigande och fallande ordning baserat på kursattribut
- Möjlighet att lägga till eller ta bort kurser från ramschemat
- Paginering med högst 50 kurser mer per klick

Ramschema-sidan ska innehålla funktionalitet för:

- Visning av antal tillagda kurser
- Beräkning av totalt antal högskolepoäng
- Ta bort kurs

Om-sidan ska innehålla:

- En hero-sektion som skapar igenkänning för målgruppen och förstärker budskapet med en bild
- En visions-sektion som beskriver Öppna Akademin's mål
- En sektion Varför välja oss som presenterar argument i form av text och ikoner i delsektionerna (cards).
- En metod-sektion som beskriver strukturen bakom plattformen
- CTA-sektion som uppmanar användaren att klicka på länken.

Kontakt-sidan ska innehålla:

- Kontaktformulär och bild

404-sidan ska innehålla:

- Emoji
- Statuskod 404 och text
- Länk tillbaka till sidan Hem

2 Teori

I detta avsnitt definieras och beskrivs de tekniker och programvaror som används under utveckling av Öppna akademins webbplats.

2.1 Angular

Angular är ett webbramverk som underhålls av ett team hos Google. Enligt den officiella dokumentationen definieras Angular som en uppsättning verktyg som underlättar utvecklingsprocessen genom att ge möjlighet av skapandet av snabba, pålitliga och skalbara webbapplikationer, både för större kodbasen och utvecklingsteam.

En av fördelarna med att använda Angular i utvecklingen av Öppna Akademin var den komponentbaserade arkitekturen, vilket underlättade återanvändning av struktur, stil och logik. Detta gjorde det möjligt att utveckla en del av applikationen i taget och samtidigt skapa en mer organiserad kod. [4].

2.1.1 Komponenter

Angular dokumentationen definierar en komponent som uppbyggd av tre huvuddelar: en HTML-template som beskriver strukturen och vad som renderas i DOM, en CSS-selector som avgör hur komponenten används i HTML samt en TypeScript-klass som hanterar komponentens logik och beteende.

En viktig fördel med komponenter är att de kan återanvändas i flera delar av applikationen, vilket bidrog till en effektivare utvecklingsprocess i Öppna Akademin. Dessutom gör Angulars data binding att DOM uppdateras automatiskt när data förändras, vilket minskar behovet av direkt manipulation av DOM jämfört med om JavaScript skulle användas [5].

2.1.2 Data binding

Angular Minds beskriver data binding som en brygga mellan HTML-templaten och TypeScript-klassen där applikationens logik definieras. Dataflödet kan ske i en eller båda riktningarna och när data förändras uppdaterar Angular automatiskt DOM.

Data kan flöda från komponentens .ts-klass till HTML-templaten, vilket kallas one-way data binding.

{{ interpolation }} används för att visa data direkt i HTML-templaten. När värdet på en variabel eller signal ändras uppdateras innehållet automatiskt i gränssnittet. Det kan vara att visa ett kursnamn eller antalet högskolepoäng.

[property binding] används för att binda egenskaper i HTML-element

till data i komponenten. Detta kan exempelvis användas för att dynamiskt ändra en bildkälla eller aktivera och inaktivera knappar beroende på applikationens tillstånd.

Data kan även flöda från HTML-templaten till komponenten.

(event binding) används för att hantera interaktioner. När en användare exempelvis klickar på en knapp eller skriver i ett formulär skickas information till komponenten där logiken med värdet kan hanteras.

[(two way data binding)] dataflöde i båda riktningarna och används oftast tillsammans med formulärfält. När användaren skriver i ett inputfält uppdateras komponentens data direkt samtidigt som förändringar i komponenten automatiskt visas i gränssnittet.

Dessa tekniker bidrog till en effektiv utveckling av Öppna Akademin. Angulars data binding gjorde det möjligt att fokusera på webbplatsens funktionalitet och användargränssnitt, samtidigt som ramverket automatiskt hanterade uppdateringar av data och vyer (DOM) [6].

2.1.3 Services och dependency injection

Enligt Angulars dokumentation används dependency injection för att organisera och återanvända kod genom att beroenden levereras utifrån istället för att skapas direkt i komponenten. Detta bidrar till bättre modularitet, skalbarhet och underhåll i applikationen.

I Angular används services för att samla logik och funktionalitet som behöver delas mellan flera komponenter. En service är en TypeScript-klass. Services kan exempelvis användas för datahantering, API-anrop, state management eller återanvändbara hjälpfunktioner.

Dependency injection innebär att Angular automatiskt kan injicera en service i en komponent utan att komponenten själv behöver skapa instansen. Detta skapar en tydligare separation mellan applikationens logik och användargränssnitt [7].

Detta medförde stora fördelar i utvecklingen av Öppna Akademin, särskilt för Kurser- och Ramschema-sidorna där samspelet mellan data och logik i CourseService och StudyPlanService effektiviserar hanteringen av filtrering, sortering och uppdatering av localStorage.

2.1.4 HTTP

Angular erbjuder HttpClient för kommunikation och hämtning av data över HTTP-protokollet. I Öppna Akademin användes HttpClient i CourseService för att hämta kursdata från filen courses.json. Den hämtade datan användes därefter för filtrering, sortering och hantering av kurser i ramschemat.

En fördel med HttpClient är att data kan hämtas asynkront utan att sidan behöver laddas om. En annan fördel är att Angular erbjuder typade svar med Interfaces, vilket bidrar till säkrare och mer strukturerad kod [8].

2.1.5 Routing

Routing i Angular används för navigering mellan olika vyer i en single page application. Routes definierar vilken komponent som ska visas för en specifik URL utan att sidan behöver laddas om.

I Öppna Akademin definierades routing i filen app.routes.ts för samtliga sidor på webbplatsen. Detta gjorde navigeringen mellan webbsidorna snabb och bidrog till en mer användarvänlig applikation [9].

2.2 Övriga programvaror

Nedanstående programvaror användes för att utveckla webbplatsen med Angular samt för att optimera och redigera bilder som användes i applikationen.

2.2.1 Visual studio code

Visual Studio Code användes som kodredigerare för utvecklingen av Öppna Akademin. Programmet användes för att skriva, strukturera och köra Angular-applikationen samt att hantera projektets filer och terminalkommandon.

2.2.2 Squoosh

Squoosh användes för att komprimera bilder. Detta bidrog till mindre filstorlekar och förbättrad laddningstid för applikationen.

2.2.3 Remove.bg

Remove.bg användes för att ta bort bakgrunden från en bild på kontakt-sidan. Detta gjorde det möjligt att bättre anpassa bilden till webbplatsens design och färgtema.

3 Metod

I detta avsnitt beskrivs arbetsmetoden, planeringsprocesser som användes vid utvecklingen av Öppna Akademin webbplats. Avsnittet behandlar hur webbplatsens struktur, design och funktionalitet planerades för att skapa en responsiv webbapplikation.

3.1 Förarbete

Flera svenska universitets webbplatser och Antagning.se analyserades

under projektets förarbete. Fokus låg på att studera struktur, språkstil och ton samt hur funktioner för filtrering, sökning och sortering användes i praktiken. Detta bidrog till planeringen av Öppna Akademinns struktur, innehåll och funktionalitet.

3.1.1 Språkstil och ton

Språkstil och ton för webbplatsens innehåll definierades genom analys av svenska universitetswebbplatser och utifrån Öppna Akademinns slogan "Gratis utbildning, för alla överallt". Målet var att skapa ett språk som upplevs professionellt, positivt och motiverande för målgruppen.

För att göra webbplatsens språk enklare att komma ihåg sammanfattades språkstilen och ton med nyckelorden positiv och möjlighet. Dessa användes som vägledning för webbplatsens texter och innehåll.

3.1.2 Design

Webbplatsens header och globala navigering skissades för både desktop och mindre skärmstorlekar för att tidigt planera navigering och responsivt beteende. Sidfoten inspirerades av Linnéuniversitetet och skissades för desktop, tablet och mobil. Dessa komponenter planerades för användning på samtliga sidor, med undantag för 404-sidan.

Sektionerna i webbplatsens huvudinnehåll (main) skissades översiktligt för att skapa en struktur.

Hem-sidan skissades med en hero-sektion innehållande rubrik, text, CTA-länk och bild, följt av en introduktions-sektion som beskriver plattformen. Därefter planerades en funktions-sektion med cards i grid-layout för funktioner såsom kurser, ramschema, sökning och filtrering samt flexibel utbildning. Varje card kompletterades med ikoner för ökad visuell tydlighet. Sidan avslutades med en CTA-sektion och en trovärdighets-sektion för att stärka den akademiska trovärdigheten.

Kurser- och Ramschema-sidorna inspirerades främst av kursens presentation (Mattias) och fokuserade främst på strukturering av sidorna.

Om-sidan planerades med en hero-sektion för att skapa igenkänning och presentera problemområdet, en visions-sektion som beskriver Öppna Akademinns mål samt sektionerna "Varför välja oss" och "Vår metod", där information presenterades genom cards med rubriker, text och ikoner. Även denna sida avslutades med en CTA-sektion.

Kontakt-sidan planerades med ett kontaktformulär innehållande fälten namn, e-post och meddelande och en knapp för att skicka formuläret

(simulerar endast). Sidan kompletterades även med en bild för att skapa ett mer visuellt tilltalande gränssnitt.

Webbplatsens färgtema inspirerades av språkstil, ton och årstiden sommar. Färgerna definierades globalt i styles.css för att skapa ett globalt färgsystem för hela applikationen.

```
--bg: #FAFAFA;
--text: #000000;
--text-secondary: #313131;
--primary: #cae6ff;
--card: #FFFFFF;
--border: #E0E0E0;
}
```

3.1.3 Definiering av innehåll

Utifrån föregående planeringsfas definierades innehållet för webbplatsens olika sektioner. Rubriker och brödtext skrevs med fokus på att följa den tidigare definierade språkstilen och tonen.

För varje sektion som använder bild definierades även vilken typ av bild som skulle användas. Detta gjordes genom beskrivande nyckelord för att underlätta sökning efter bilder som visuellt förstärker sektionens innehåll.

3.1.4 Bilder och ikoner

Designskissen fungerade som underlag för val av bilder som valdes för att visuellt förstärka webbplatsens innehåll och budskap. Bilderna optimerades med Squoosh innan de placerades i projektets public mapp i det initierade Angular projektet.

Ikonerna hämtades i SVG-format för god skalbarhet vid olika skärmstorlekar. Dessa bearbetades inte i något bildprogram, utan användes i originalformat för att bibehålla kvalitet.

3.2 Planering av struktur och design

Webbplatsens struktur och layout planerades och implementerades initialt med HTML och CSS, innan någon logik applicerades. Detta

gjordes för att skapa en grundstruktur och ett visuellt gränssnitt innan funktionalitet programmerades.

3.2.1 Responsiv design

Utvecklingen påbörjades för desktop, där layout och sektioner successivt anpassades för mindre skärmstorlekar. Detta arbetssätt möjliggjorde en stegvis anpassning av gränssnittet innan logiken applicerades.

3.2.2 Mappstruktur

Efter initiering av projektet med ng new organiserades projektets filstruktur för att skapa en utgångspunkt.

Utifrån designskisser skapades mappar i public för statiska resurser som bilder, ikoner och JSON-fil, uppdelat i mappar för respektive typ.

I src/app strukturerades koden i separata mappar för att skapa en separation mellan olika delar av applikationen. Här skapades bland annat components (cta-link, footer, header, image och logo), pages (home, about, contact, course, study-plan och not-found), models för interfaces och services för applikationens logik och hantering av data.

Denna struktur valdes för att underlätta återanvändning av komponenter, skapa en tydlig uppdelning mellan presentation, logik och data.

3.2.3 Komponentstruktur

Header och footer återanvänds på samtliga sidor, med undantag för 404-sidan. Dessa importerades i app.ts och placerades i app.html för att skapa en konsekvent layout och navigering genom hela applikationen.

Övriga komponenter i components-mappen användes vid behov och återanvändes på olika sidor för att minska upprepning av kod.

3.2.4 Interface

Interface definierades utifrån JSON-filen och attributen för kurs samt placerades in i models-mappen:

```
export interface CourseInterface {
  courseCode: string;
  subjectCode: string;
  level: string;
  progression: string;
  courseName: string;
  points: number;
  institutionCode: string;
  subject: string;
  syllabus: string;
}
```

3.3 Planering av funktionalitet

Applikationens routing definierades i `app.routes.ts`, där varje sida kopplades till en specifik URL. En fallbackrutt implementerades för att hantera felaktiga sökvägar, vilket leder användaren till 404-sidan.

I `app.html` användes en `@if` sats för att dölja header med global navigering och footer på 404-sidan, för att skilja denna sida från resterande delar av webbplatsen.

3.3.1 Definiering av funktionalitet

Vid inladdning av Kurser-sidan visas initialt 50 kurser, med möjlighet att visa upp till 50 ytterligare per klick på med "visa fler kurser"-knappen.

Filtrering implementeras både globalt på kurskod eller kursnamn och inom ett valt ämnesområde, där ämnesområde även kan väljas utan att övriga filter är aktiva.

Sortering kan utföras på kursattribut i både stigande och fallande ordning.

Användaren ges även möjlighet att lägga till eller ta bort kurser från ramschemat direkt från Kurser-sidan.

På Ramschema-sidan finns funktionalitet för att ta bort kurser från tabellen. Sidan visar även antal tillagda kurser samt den totala summan av högskolepoäng.

3.3.2 Services och localStorage

Services skapades utifrån ansvar, där Kurser-sidan och Ramschema-sidan tilldelades varsin separat service. Metoderna i respektive service definierades utifrån den tidigare specificerade funktionaliteten och hur samt när de skulle användas i applikationen.

För lagring av data användes localStorage för att spara valda kurser för Ramschema-sidan. För att garantera att både kursdata och sparade kurser var tillgängliga direkt vid applikationens start valdes att initiera metoder från respektive service i app.ts, vilket gjorde datan globalt tillgänglig.

3.4 Versionshantering

För att hantera versionshistorik av kod användes Git tillsammans med GitHub.

3.5 Arbetsmetod

Implementeringen av webbplatsen genomfördes iterativt, där utveckling, felsökning och förbättring skedde stegvis.

Webbplatsen testades med jämna mellanrum i webbläsaren Google Chrome för att testa funktionalitet, responsivitet och korrekt rendering av komponenter.

4 Konstruktion

I detta avsnitt beskrivs konstruktionen av Öppna Akademin webbplats. Fokus ligger på hur applikationen har strukturerats med Angular, hur routing, komponenter, services och datahantering har implementerats för att skapa en modulär webbapplikation.

Utvecklingen utgår från en komponentbaserad arkitektur där separation av ansvar har varit en princip, i enlighet med Angulars arbetssätt.

4.1 Routing och applikationsstruktur

Routing implementerades i app.routes.ts och används för att hantera navigering mellan applikationens olika vyer utan omladdning av sidan. Detta följer principen för single-page applications där klientbaserad routing används för att skapa en snabb växling mellan webbplatsens sidor.

En rutt har implementerats för att hantera ogiltiga URL:er, vilket leder

användaren till en 404-sida. För att se till att en separation av webbplatsens undersidor döljs global navigation och sidfot på 404-sidan via if-sats i app.html.

4.2 Komponentbaserad arkitektur

Webbplatsen är uppbyggd enligt Angulars komponentbaserade modell där varje komponent har ett avgränsat ansvar. Header och footer är globala komponenter som återanvänds genom hela applikationen, vilket bidrar till en konsekvent användarupplevelse och minskad duplicering av kod.

Övriga sidor innehåller komponenter som återanvändbara byggblock såsom knappar, bilder och call-to-action-länkar. Denna struktur ger möjligheten att återanvändbarhet stycken av UI-element för att både snabba på utvecklingsprocessen och minimera duplicering.

4.3 Services och datahantering

Webbplatsen använder services för att möjliggöra global återanvändning av logik. Kurser-sidans funktionalitet hanteras i en separat service, medan Ramschema-sidans hanteras i en egen service. Detta skapar en ansvarsfördelning och ger möjlighet för komponenter att dela logik med varandra.

Hantering av applikationens tillstånd (state) sker via services i kombination med localStorage, vilket gör det möjligt att bevara användarens val mellan sessioner. Detta val gjordes för att undvika behovet av en backend-lösning, eftersom persistens av data uppnås på klientsidan. Även data binding används tillsammans med services logik för att via signal eller variabler uppdatera vyn dynamiskt via ändring av definierade värden.

Services injiceras i komponenter via dependency injection, vilket är ett rekommenderat tillvägagångssätt i Angular version 21 och bidrar till global delning av logik.

4.4 Funktionalitet (filtrering, sortering och paginering)

Kurser-sidan är uppbyggd kring funktioner för filtrering, sortering och paginering av kursdata. Vid initial laddning visas 50 kurser och användaren kan successivt ladda fler i steg om 50.

Filtrering kan utföras både globalt (genom sökning av kurskod eller kursnamn) eller baserat på ämnesområde. Detta skapar flexibel sökning utan att begränsa användaren till en specifik filterlogik.

Sortering implementeras för olika kursattribut i både stigande och fallande ordning, vilket ger användaren möjlighet att lista kurser i bokstavsordning eller omvänd bokstavsordning.

Användaren kan även lägga till och ta bort kurser direkt från tabellen för

att skapa ramschema, vilket uppdaterar ramschemat med hjälp av delad logik från CourseService och StudyPlanService.

4.5 Ramschema och beräkningar

Ramschema-sidan visar användarens valda kurser och fungerar som en sammanställning av studieplanen. Här implementeras funktionalitet för att ta bort kurser och för att dynamiskt uppdatera innehållet.

Logik för att beräkna totalt antal kurser och sammanlagda högskolepoäng. Denna logik är global via services för att möjliggöra hantering av data.

4.6 Datamodeller och typning

För en konsekvent och säker datastruktur används ett interface för att definiera kursobjekt i JSON-filen. Detta minskar risken för felaktig datahantering och gör koden mer förutsägbar.

Typning används genom hela applikationen, vilket bidrar till robustare kod samt minskar antalet fel i programmen.

4.7 Responsiv design och användargränssnitt

Webbplatsen är utvecklad med responsiv design där layout och komponenter anpassas efter olika skärmstorlekar. Detta gör att applikationen går att använda på desktop, tablet och mobil.

Arbetet har utgått från en desktop-first anpassning där layout successivt justerats för mindre skärmar, eftersom gränssnittet inte var allt för komplicerat i sin struktur.

5 Resultat

Resultatet av projektet blev en responsiv webbapplikation utvecklad med Angular som uppfyller projektets grundläggande krav för Öppna Akademiens webbplats. Den består av sidorna Hem, Kurser, Ramschema, Om, Kontakt och en 404-sida för hantering av felaktiga URL:er.

Webbplatsen implementerades med global navigering och sidfot som återanvänds mellan samtliga huvudsidor. Layout och innehåll anpassas för olika skärmstorlekar genom responsiv design för desktop, tablet och mobil.

Kurser-sidan implementerades med funktionalitet för filtrering baserat på kurskod, kursnamn och ämnesområde och sortering i stigande samt fallande ordning. Sidan innehåller även paginering där användaren successivt kan visa fler kurser.

Användaren kan lägga till och ta bort kurser från ramschemat direkt från Kurser-sidan. För att minska duplicering av data sparas endast kursens id i localStorage istället för hela kursobjekt. Vid initiering av applikationen används en new Map (datastruktur) för att koppla sparade id:n mot motsvarande kursobjekts id:n från JSON-datan, vilket ger effektivare datahantering, snabb tillgång till data och minskad redundant lagring av kursinformation.

Ramschema-sidan visar användarens valda kurser, totalt antal kurser och sammanlagda högskolepoäng. Uppdateringar sker dynamiskt när kurser läggs till eller tas bort från ramschemat utan att sidan laddas om.

Kontakt-sidan implementerades med ett kontaktformulär innehållande namn, e-post och meddelande. Formuläret är inte kopplat till någon backend eller extern tjänst för hantering av skickade meddelanden.

Den komponentbaserade strukturen tillsammans med services, routing och typade interfaces bidrog till en modulär och strukturerad webbplats där presentation, logik och datahantering separerades.

6 Slutsatser

Projektets resultat visar att det är möjligt att utveckla en responsiv och modulär webbplats med Angular som uppfyller kraven för Öppna Akademin. Genom användning av komponentbaserad arkitektur, routing, services och localStorage skapades en webbplats med funktionalitet för filtrering, sortering, paginering och hantering av ramschema.

Den valda strukturen med separata services för kursdata och ramschema bidrog till ansvarsfördelning och bättre separation mellan logik och presentation. Användningen av new Map tillsammans med sparade kurs-id:n bidrog till effektivare datahantering genom att undvika duplicering av kursobjekt från JSON-filen och direkt slå upp kursdata.

Projektet visar även hur responsiv design och återanvändbara komponenter kan användas för att skapa en mer konsekvent användarupplevelse för olika skärmstorlekar.

Det finns samtidigt delar av webbplatsen som hade kunnat förbättras. Viss kod hade kunnat refaktorerats för att minska duplicering av kod. Exempelvis hade funktionalitet för sortering i stigande och fallande ordning kunnat kombineras i mer generella metod.

Användarupplevelsen hade även kunnat förbättras. När användaren navigerar till Kurser-sidan hade applikationen kunnat återställa scroll positionen till sidans topp. En separat scrolla till toppen funktionalitet hade kunnat implementeras för att underlätta navigering när användaren scrollat långt ner på sidan.

Kontaktformuläret implementerades som frontend funktionalitet och saknar koppling till backend eller extern tjänst för hantering av skickade meddelanden. Formuläret hade kunnat integreras med exempelvis ett API eller en tjänst för att skapa fullständig funktionalitet.

Arbetet med Angular upplevdes positivt eftersom ramverket bygger på strukturerat och logiskt tänkande. Den komponentbaserade arkitekturen, användningen av services och dataflöden och uppdelningen mellan presentation och logik påminde till viss del om matematiskt problemlösande där större problem delas upp i mindre och mer hanterbara delar. En annan fördel var att Angular automatiskt hanterar uppdatering av DOM genom data binding, vilket minskade utvecklingsstegen, att skapa referenser till DOM och uppdatera den som i vanilla JavaScript.

Källförteckning

- [1] World Population Review, "Countries With Free College 2026"
<https://worldpopulationreview.com/country-rankings/countries-with-free-college> Publicerad datum saknas. Hämtad 2026-05-13.
- [2] Worldometer, "How many countries are there in the world 2026"
<https://www.worldometers.info/geography/how-many-countries-are-there-in-the-world/> Publicerad datum saknas. Hämtad 2026-05-13.
- [3] University of the People, "10 benefits of education that will surprise you",
<https://www.uopeople.edu/blog/benefits-of-education-are-societal-and-personal/> Publicerad 2025-08-11. Hämtad 2026-05-13.
- [4] angular.dev "What is angular" <https://angular.dev/overview> Publicerad datum saknas. Hämtad 2026-05-13.
- [5] angular.dev "Anatomy of a component"
<https://angular.dev/guide/components> Publicerad datum saknas. Hämtad 2026-05-13.
- [6] angularminds.com, "Data binding in Angular:everything you need to know". <https://www.angularminds.com/blog/data-binding-in-angular> Publicerad 2024-04-10. Hämtad 2026-05-13.
- [7] angular.dev "Dependency injection:overview" <https://angular.dev/guide/di> Publicerad datum saknas. Hämtad 2026-05-13.
- [8] angular.dev "HTTP Client" <https://angular.dev/guide/http> Publicerad datum saknas. Hämtad 2026-05-13.
- [9] angular.dev "Angular routing" <https://angular.dev/guide/routing> Publicerad datum saknas. Hämtad 2026-05-13.